



贪心算法

——一种求解最优化问题的有效算法

朱容波

Email: rongbozhu@126.com

Office: 一综B323

华中农业大学信息学院



本章内容

- 引言
- 最短路径问题
- 最小耗费生成树
 - Kruskal算法
 - Prim算法
- 文件压缩

华中农业大学信息学院



8.1 引言

- 顾名思义，**贪心算法**总是作出在当前看来最好的选择。也就是说贪心算法并不从整体最优考虑，它所作出的选择只是在某种意义上的**局部最优**选择。当然，希望贪心算法得到的最终结果也是整体最优的。
- 虽然贪心算法不能对所有问题都得到整体最优解，但对许多问题它能产生整体最优解。如单源最短路径问题，最小生成树问题等。
- 在一些情况下，即使贪心算法不能得到整体最优解，其最终结果却是**最优解的很好近似**。

华中农业大学信息学院



8.1 引言

- 与动态规划算法比较:
 - 贪心算法通常用来求解最优化问题，即量的最大化或最小化。
 - 然而，贪心算法不像动态规划算法，它通常包含一个用以寻找局部最优解的迭代过程。在某些实例中，这些局部最优解转变成了全局最优解，而在另外一些情况下，则无法找到最优解。

华中农业大学信息学院



基本思想

- 贪心算法在少量计算的基础上做出正确猜想而不急于考虑以后的情况。
- 它一步步地来构筑解。
- 每一步均是建立在局部最优解的基础之上，而每一步又都扩大了部分解的规模。
- 做出的选择产生最大的直接收益而又保持可行性。
- 因为每一步的工作很少且基于少量信息，所得算法特别有效。

华中农业大学信息学院



例子: 分数背包问题

? 问题描述:

- 给出 n 个大小为 $s_1, s_2, \dots, s_n$, 值为 $v_1, v_2, \dots, v_n$ 的项目, 并设背包容量为 C , 要找到非负实数 $x_1, x_2, \dots, x_n$ 使和

$$\sum_{i=1}^n x_i v_i$$

- 在约束

$$\sum_{i=1}^n x_i s_i \leq C$$

- 下最大

华中农业大学信息学院



例子

- $n=3, C=20, (v_1, v_2, v_3)=(25, 24, 15),$
 $(s_1, s_2, s_3)=(18, 15, 10)$

- 可行方案:

(x_1, x_2, x_3)	$\sum s_i x_i$	$\sum v_i x_i$
(1)(1/2, 1/3, 1/4)	16.5	24.5
(2)(1, 2/15, 0)	20	28.2
(3)(0, 2/3, 1)	20	31
(4)(0, 1, 1/2)	20	31.5

华中农业大学信息学院



选择策略-1

- 从剩下的物品中选择最大价值的物品装入背包，得到：
- 方案(2)(1,2/15,0) 20 28.2 (非最优)
- 原因：虽然价值很大，但是容量消耗的太快

华中农业大学信息学院



选择策略-2

- 从剩下的物品中选择最小尺寸的物品装入背包。
- 方案(3) (0,2/3,1) 20 **31** (非最优)
- 原因：虽然容量损耗较少，但是价值增长速度太慢

华中农业大学信息学院



选择策略-3

- 从剩下的物品中选择最大 v_i/s_i 比率的物品装入背包。
- Solution (4) (0,1,1/2) 20 31.5 (最优)

华中农业大学信息学院



Greedy algorithm to solve Knapsack

- Step 1:
 - 每项计算 $y_i = v_i/s_i$, 即该项值和大小的比
- Step 2:
 - 再按比值的降序来排序, 从第一项开始装背包, 然后是第二项, 依次类推, 尽可能地多放, 直至装满背包。

华中农业大学信息学院



算法Greedy-KNAPSACK

输入: 按照 $v(i)/s(i)$ 比率升序排列的 n 个物品的 容量数组 $s[1..n]$ 和物品价值数组 $v[1..n]$; 背包的总容量 C , 物品的数量 n

输出: 最优贪心算法结果 $x[1..n]$

```
for  $i \leftarrow 0$  to  $n$     {初始化 $x[i]$  }  
     $x[i] \leftarrow 0$   
end for  
 $cu \leftarrow C$   
for  $i \leftarrow 0$  to  $n$     {物品按次序装入背包 }  
    if  $s[i] > cu$  then  
        exit  
    end if  
     $x(i) \leftarrow 1$   
     $cu \leftarrow cu - s(i)$  {剩余背包容量}  
end for  
if  $i \leq n$  then  $x(i) \leftarrow cu/s(i)$  end if {最后剩余背包容量中装入物品 $i$ 的比率}  
return  $x$ 
```

华中农业大学信息学院



贪心算法架构

```
Algorithm GREEDY(A,n)
  solution  $\leftarrow \emptyset$ 
  for i  $\leftarrow$  1 to n do
    x  $\leftarrow$  SELECT(A)
    if FEASIBLE(solution,x)
      then solution  $\leftarrow$  UNION(solution,x)
    end if
  end for
  return (solution)
```

华中农业大学信息学院



贪心算法特点

- 算法由一个简单的迭代过程构成，在维持可行性的前提下它选择能产生最大直接利益的项。
- 贪心算法产生全局最优解的最重要因素是选择策略。

华中农业大学信息学院



贪心算法基本要素

- 对于一个具体的问题，怎么知道是否可用贪心算法解此问题，以及能否得到问题的最优解呢？这个问题很难给予肯定的回答。
- 但是，从许多可用贪心算法求解的问题中我们看到这类问题一般具有2个重要的性质：**贪心选择性质**和**最优子结构性质**。

华中农业大学信息学院



贪心选择性质

- 所谓贪心选择性质是指所求问题的整体最优解可以通过一系列局部最优的选择，即贪心选择来达到。这是贪心算法可行的第一个基本要素，也是贪心算法与动态规划算法的主要区别。
- 动态规划算法通常以自底向上的方式解各子问题，而贪心算法则通常以自顶向下的方式进行，以迭代的方式作出相继的贪心选择，每作一次贪心选择就将所求问题简化为规模更小的子问题。
- 对于一个具体问题，要确定它是否具有贪心选择性质，必须证明每一步所作的贪心选择最终导致问题的整体最优解。

华中农业大学信息学院



最优子结构性质

- 当一个问题最优解包含其子问题的最优解时，称此问题具有**最优子结构性质**。问题的最优子结构性质是该问题可用动态规划算法或贪心算法求解的关键特征。
- 贪心算法和动态规划算法都要求问题具有最优子结构性质，这是2类算法的一个共同点。但是，对于具有最优子结构的问题应该选用贪心算法还是动态规划算法求解？是否能用动态规划算法求解的问题也能用贪心算法求解？

华中农业大学信息学院



8.2 最短路径问题

问题描述:

- 设 $G = (V, E)$ 是一个每条边有非负长度的有向图，有一个特异顶点 s 称为源。
- 单源最短路径问题，或者简称为最短路径问题，是要确定从 s 到 V 中每一个其他顶点的距离，这里从顶点 s 到顶点 x 的距离定义为从 s 到 x 的最短路径的长度。

华中农业大学信息学院



Dijkstra算法基本思想

贪心技术: Select the path in nonincreasing order.

- 为简便起见, 我们假设 $V = \{1, 2, \dots, n\}$, $s=1$.
- 初始时, 将顶点分为两个集合 $X = \{1\}$ 和 $Y = \{2, 3, \dots, n\}$ 。这样做的目的是让 X 包含这样的顶点集合: 从源点到这些顶点的距离已经确定。
- 在每一步中, 我们选定源点到它的距离已经获得的一个顶点 $y \in Y$, 并将这个顶点移到 X 中。
- 与 Y 中的每个顶点 y 联系的是标记 $\lambda[y]$, 它是只经过 X 中顶点的最短路径的长度, 一旦顶点 $y \in Y$ 移到 X 中, 与 y 相邻的每个顶点 $w \in Y$ 的标记就被更新, 这表示找到了经过 y 到 w 的更短路径。

华中农业大学信息学院



算法8.1 DIJKSTRA

输入: 含权有向图 $G=(V,E)$, where $V=\{1,2,\dots,n\}$.

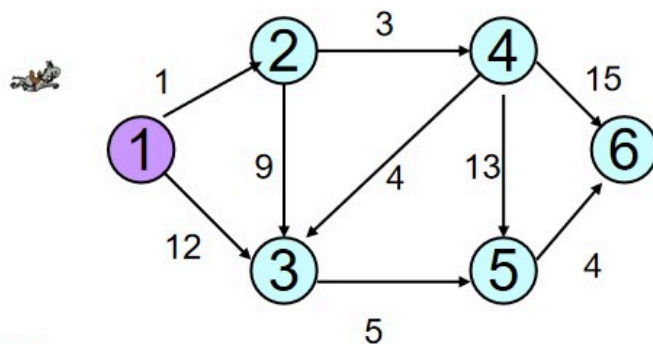
输出: G 中顶点 1 到其他顶点的距离。

```
1.  $X=\{1\}$ ;  $Y \leftarrow V-\{1\}$ ;  $\lambda[1] \leftarrow 0$ 
2. for  $y \leftarrow 2$  to  $n$ 
3.     if  $y$  相邻于 1 then  $\lambda[y] \leftarrow \text{length}[1,y]$ 
4.     else  $\lambda[y] \leftarrow \infty$ 
5.     end if
6. end for
7. for  $j \leftarrow 1$  to  $n-1$ 
8.     令  $y \in Y$  使得  $\lambda[y]$  最小
9.      $X \leftarrow X \cup \{y\}$            {将顶点  $y$  加入  $X$ }
10.     $Y \leftarrow Y - \{y\}$          {将顶点  $y$  从  $Y$  中删除}
11.    for 每条边  $(y,w)$ 
12.        if  $w \in Y$  and  $\lambda[y] + \text{length}[y,w] < \lambda[w]$  then
13.             $\lambda[w] \leftarrow \lambda[y] + \text{length}[y,w]$ 
14.        end for
15. end for
```

华中农业大学信息学院



例子



X
 {1}
 {1,2}
 {1,2,4}
 {1,2,4,3}
 {1,2,4,3,5}
 {1,2,4,3,5,6}

Success!

Y

2	3	4	5	6
1	12	∞	∞	∞

3	4	5	6
10	4	∞	∞

3	5	6
8	17	19

5	6
13	19

6
17

华中农业大学信息学院



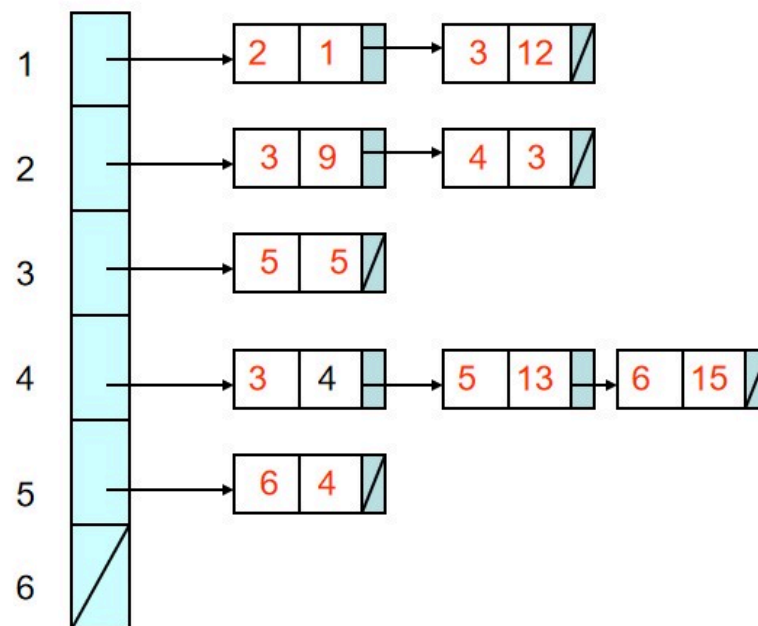
数据结构

X:

1	2	3	4	5	6
T	F	F	F	F	F

Y:

1	2	3	4	5	6
F	T	T	T	T	T



华中农业大学信息学院



正确性



引理8.1:

- 在算法DIJKSTRA中, 当顶点 y 在第8步中被选中, 如果标记 $\lambda[y]$ 是有限的, 那么、 $\lambda[y] = \delta[y]$ 。($\delta[y]$ 表示从源点 x 到 y 的距离)
- 证明:
 - 对顶点离开集合 Y 并进入 X 的次序做归纳。
 - 第1个离开的顶点是1, 我们有 $\lambda[1] = \delta[1] = 0$ 。假设引理中的结论对于所有在 y 前离开 Y 的顶点都成立。
 - 因为 $\lambda[y]$ 是有限的, 则必存在从1到 y 的路径, 它的长是 $\lambda[y]$ 。设 $\pi = 1, \dots, x, w, \dots, y$ 是从1到 y 的最短路径, 其中 x 是在 y 前最迟离开 Y 的顶点。我们有 $\lambda[y] \leq \lambda[w] \leq \lambda[x] + \text{length}(x, w) = \delta[x] + \text{length}(x, w) = \delta[w] \leq \delta[y]$ 。
 - 上述的证明基于这样一个假设, 即所有的边长都是非负的。
- p149

华中农业大学信息学院



算法分析

- 第1步: $\Theta(n)$ 时间.
- 第3步 和第4步: 分别为 $\Theta(n)$ 和 $O(n)$
- 第8步: $\Theta(n^2)$.
- 第9步和第10步: 每次迭代花费时间 $\Theta(1)$, 公用了 $\Theta(n)$ 时间.
- 第11步和第12步共需要时间 $\Theta(m)$. 这里 $m=|E|$.
- 根据以上得出算法的时间复杂度是 $\Theta(m+n^2) = \Theta(n^2)$.
- p149



定理 8.1

- 给出一个边具有非负权的有向图 G 和源顶点 s , 算法 DIJKSTRA 在时间 $\Theta(n^2)$ 内找出从 s 到其他每一顶点距离的长度

华中农业大学信息学院



8.2.1 稠图的线性时间算法

- 基本, :
 - 用最小堆数据结构来保持集合Y中的顶点, 使Y组中离V-Y最近的顶点y可以在 $O(\log n)$ 时间内被选出
 - 与每个顶点v相关的键就是它的标记 $\lambda[v]$.
 - 最后的算法如算法SHORTESTPATH所示

华中农业大学信息学院



算法8.2 SHORTESTPATH

输入: 含权有向图 $G=(V,E)$, where $V=\{1,2,\dots,n\}$.

输出: G 中顶点1到其他顶点的距离。假设已有一个空堆 H 。

1. $Y \leftarrow V - \{1\}$; $\lambda[1] \leftarrow 0$; $\text{key}(1) \leftarrow \lambda[1]$

2. **for** $y \leftarrow 2$ **to** n

3. **if** y 邻接于1 **then**

4. $\lambda[y] = \text{length}[1,y]$

5. $\text{key}(y) \leftarrow \lambda[y]$

6. $\text{INSERT}(H,y)$

7. **else**

8. $\lambda[y] \leftarrow \infty$

9. $\text{key}[y] \leftarrow \lambda[y]$

10. **end if**

11. **end for**

NEXT PAGE

华中农业大学信息学院



算法8.2 SHORTESTPATH

```
12. for j ← 1 to n-1
13.     y ← DELETEMIN(H)
14.     Y ← Y - {y}    {将顶点y从Y删除}
15.     for 每个邻接于y的顶点 w ∈ Y
16.         if  $\lambda[y] + \text{length}[y, w] < \lambda[w]$  then
17.              $\lambda[w] \leftarrow \lambda[y] + \text{length}[y, w]$ 
18.              $\text{key}(w) \leftarrow \lambda[w]$ 
19.         end if
20.         if  $w \notin H$  then INSERT(H, w)
21.         else SIFTUP(H,  $H^{-1}(w)$ ) { $H^{-1}(w)$ 返回w在H中的位置}
22.         end if
23.     end for
24. end for
```

华中农业大学信息学院



算法分析

需要强调的:输入算法的是邻接表,如果用邻接矩阵,我们最终会得到一个 $\Theta(n^2)$ 时间的算法。

- 运行时间主要取决于堆运算。
- 这里共有 $n-1$ 个DELETEMIN运算, $n-1$ 个INSERT运算, 最多 $m-n+1$ 个SHIFTUP运算。每个堆运算用 $O(\log n)$ 时间, 得到总共需要 $O(m \log n)$ 的时间。

- 定理8.2



- 给出具有非负权重的边和源顶点 s 的图 G , 算法SHORTESTPATH可在 $O(m \log n)$ 时间内找出从 s 到其他每一个顶点的距离。如果图是稠密的, 即对于某个 $\epsilon > 0$, $m \geq n^{1+\epsilon}$, 那么它可以被改善为在时间 $O(m/\epsilon)$ 内运行。
- P151

华中农业大学信息学院



8.3 最小耗费生成树 (Kruskal算法)

定义8.1:

- 设 $G = (V, E)$ 是一个具有含权边的连通无向图。 G 的一棵生成树 (V, T) 是 G 的作为树的子图。如给 G 加权并且 T 的各边的权的和为最小值, 那么 (V, T) 就称为**最小耗费生成树**或简称为**最小生成树**

? 问题描述

- 假设 G 为连通的, 怎么找到最小生成树 (MST)?

华中农业大学信息学院



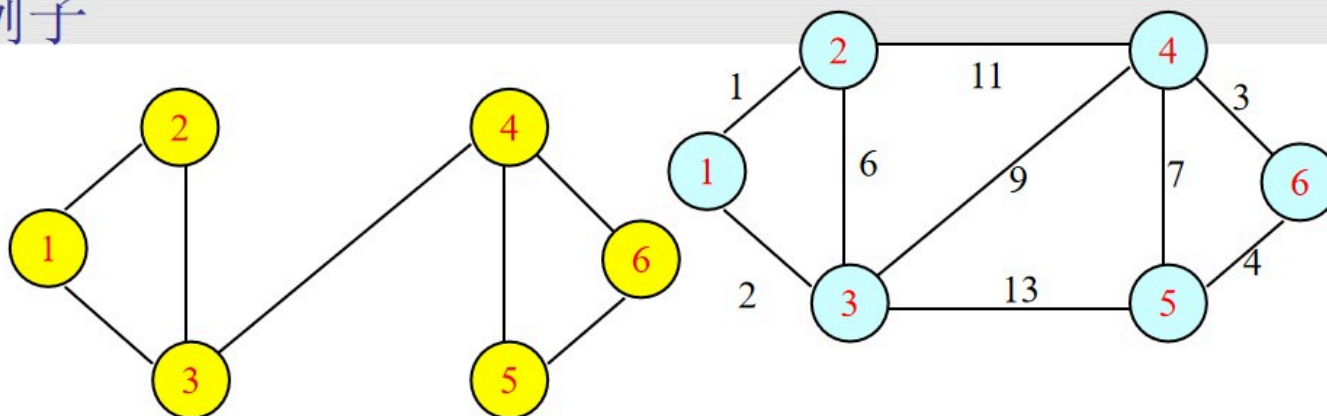
Kruskal算法基本思想

- 工作原理：维护由许多生成树组成的一个森林，这些生成树逐步合并直到最终森林只由一棵树组成，这棵树就是最小耗费生成树。
- Step 1: 此算法先从对权以非降序排列着手。
- Step 2: 接着从由图的顶点组成而不包含边的森林 (V, T) 开始，重复下面的这一步，直到 (V, T) 被变换成一棵树：
 - 设 (V, T) 是到现在为止构建的森林， $e \in E - T$ 为当前考虑的边，如果把 e 加到 T 中不生成一个回路，那么将 e 加入进 T ，否则丢弃。
 - 这个处理在恰好加完 $n-1$ 条边后结束。

华中农业大学信息学院



例子



(1,2)	(1,3)	(4,6)	(5,6)	(2,3)	(4,5)	(3,4)	(2,4)	(3,5)
1	2	3	4	6	7	9	11	13

!构成回路, 这条边被丢弃.

Success!

华中农业大学信息学院



Kruskal算法的执行

- 数据结构来表示森林:
 - 为有效地实现此算法，我们需要某种机制来检测加入边后**是否构成回路**。让它在算法的每个时刻来表示森林，并且在向T中添加边时动态检测是否有回路生成。
 - 这种数据结构的一个合适选择是4.3节讨论过的**不相交集表示法**
 - 开始时，图的每个顶点由一棵包含一个顶点的树表示
 - 在算法的执行过程中，森林中的每个连通分量由一棵树来表示。

华中农业大学信息学院



算法8.3 KRUSKAL

输入:包含 n 个顶点的含权连通无向图 $G=(V,E)$ 。

输出:

由 G 生成的最小耗费生成树 T 组成的边的集合。

1. 按非降序权重将 E 中的边排序
2. **for** 每条边 $v \in V$
3. MAKESET $\{v\}$
4. **end for**
5. $T = \{\}$
6. **while** $|T| < n-1$
7. 令 (x,y) 为 E 中的下一条边.
8. **if** FIND(x) $\neq$ FIND(y) **then** {检查 x , y 是否在同一颗树中}
9. 将 (x,y) 加入 T
10. UNION(x,y)
11. **end if**
12. **end while**

华中农业大学信息学院



正确性

🌐 引理8.2: 在含权无向图中, 算法KRUSKAL正确地找出最小生成树。

• 反证法: 假设KRUSKAL生成的不是最小生成树

- 1). 设KRUSKAL生成的树为 G_0
- 2). 假设存在 $G_{\min}$ 使得 $\text{cost}(G_{\min}) < \text{cost}(G_0)$ 则在 $G_{\min}$ 中存在 $\langle u, v \rangle$ 不属于 G_0
- 3). 将 $\langle u, v \rangle$ 加入 G_0 中可得一个环, 且 $\langle u, v \rangle$ 不是该环的最长边(这是因为 $\langle u, v \rangle \in G_{\min}$)
- 4). 这与KRUSKAL每次生成最短边矛盾
- 5). 故假设不成立, 命题得证.

Proof P241(or P153)

华中农业大学信息学院



算法分析

- 第1步和第2步分别花费 $O(m \log m)$ 和 $\Theta(n)$, 这里 $m = |E|$.
- 第7步好执行 $n-1$ 次总共要 $\Theta(n)$ 时间。合并运算执行 $n-1$ 次, 查找运算最多 $2m$ 次.
- 第5步花费 $\Theta(1)$, 再加上第9步最多执行 m 次, 它的总花费是 $O(m)$.
- 由定理4.3, 这两个运算的总花费是 $O(m \log^* n)$ 。这样算法总的运行时间取决于排序步, 也就是 $O(m \log m)$
- p153

华中农业大学信息学院



8.4 最小耗费生成树 (Prim算法)

- 基本思想
- 设 $G=(V,E)$, 为了简便起见, V 取整数集合 $\{1,2,\dots,n\}$.
- 算法从建立两个顶点集合开始: $X=\{1\}$ 和 $Y=\{2,\dots,n\}$ 。接着生长一棵生成树, 每次一条边。
- 在每一步, 它找出一条权重最小的边 (x,y) , 这里 $x \in X$, $y \in Y$ 并且把 y 从 Y 移到 X 。重复这一步直到 Y 为空。

华中农业大学信息学院



Prim算法

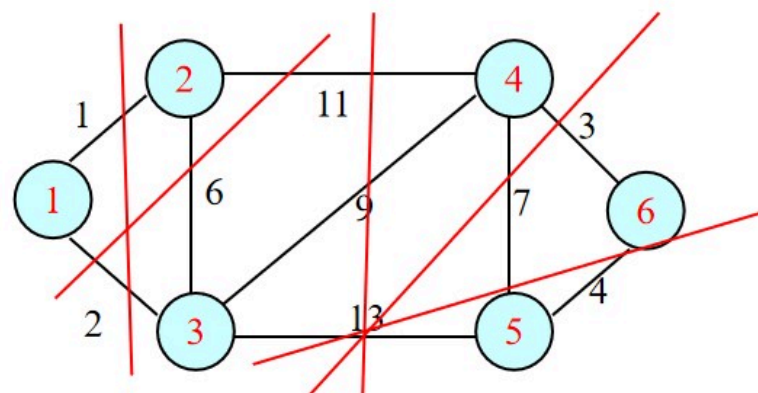
此算法概况如下:

1. $T \leftarrow \{\}$; $X \leftarrow \{1\}$; $Y \leftarrow V - \{1\}$
2. **while** $Y \neq \{\}$
3. 设 (x,y) 是最小权重的边, 其中 $x \in X$ and $y \in Y$
4. $X \leftarrow X \cup \{y\}$
5. $Y \leftarrow Y - \{y\}$
6. $T \leftarrow T \cup \{(x,y)\}$
7. **end while**

华中农业大学信息学院



例子



Success!

华中农业大学信息学院



算法 8.4 PRIM p154

输入:含权连通无向图 $G=(V,E)$, 其中 $V=\{1,2,\dots,n\}$.

输出:由 G 生成的最小耗费生成树 T 组成的边的集合。

```
1.  $T \leftarrow \{\}$ ;  $X \leftarrow \{1\}$ ;  $Y \leftarrow V - \{1\}$ 
2. for  $y \leftarrow 2$  to  $n$ 
3.     if  $y$  邻接于 1 then
4.          $N[y] \leftarrow 1$ 
5.          $C[y] \leftarrow c[1,y]$ 
6.     else  $C[y] \leftarrow \infty$ 
7.     end if
8. end for
```

NEXT PAGE

华中农业大学信息学院



算法 8.4 PRIM

```
9. for j ← 1 to n-1 {寻找n-1 条边}
10.     令  $y \in Y$  使得  $C[y]$  最小
11.      $T \leftarrow T \cup \{y, N[y]\}$            {将边  $(y, N[y])$  加入  $T$ }
12.      $X \leftarrow X \cup \{y\}$              {将顶点  $y$  加入  $X$ }
13.      $Y \leftarrow Y - \{y\}$            {从  $Y$  删除顶点}
14.     for 每个邻接于  $y$  的顶点  $w \in Y$ 
15.         if  $c[y, w] < C[w]$  then
16.              $N[w] \leftarrow y$ 
17.              $C[w] \leftarrow c[y, w]$ 
18.         end if
19.     end for
20. end for
```

华中农业大学信息学院



算法分析

- 第1步耗费 $\Theta(n)$ 时间.
- 第2步的for循环需要时间 $\Theta(n)$.第3步到第6步需要时间 $O(n)$.
- 第10步搜索离X最近的顶点y, 每迭代一次需要花费时间 $\Theta(n)$, 而这一步执行了 $n-1$ 次, 所以第10步一共花费时间 $\Theta(n^2)$
- 第11步、第12步和第13步, 每迭代一次花费时间 $\Theta(1)$ 总共花费时间 $\Theta(n)$ 。
- 第14步的for循环执行了 $2m$ 次, 第14步一共需要的时间是 $\Theta(m)$
- ...P156
- 这就得到了算法的时间复杂性是 $\Theta(m+n^2)$

华中农业大学信息学院



8.4.1 稠图的线性时间算法

基本思想

- 现在要改进算法PRIM，就像曾经对算法DIJKSTRA所做的那样，目标是把 $m = o(n^2)$ 的那类图的时间复杂性从 $\Theta(n^2)$ 减少到 $O(m \log n)$ 。以后还要进一步改进它，使在稠图情况下，可以在对边数的线性时间内运行。
- 就像在算法SHORTESTPATH中那样，基本思想是用**最小堆**数据结构(见4.2节)来保持边界顶点集，使得可以在 $O(\log n)$ 时间内取得Y集中的顶点y，这个y和V-Y集中一个顶点连接的边的耗费是最小的。修改后的算法在算法MST中给出。

华中农业大学信息学院



算法 8.5 MST

输入:含权连通无向图 $G=(V,E)$, where $V=\{1,2,\dots,n\}$.

输出:由 G 生成的最小耗费生成树 T 组成的边的集合。假设我们已有一个空堆 H .

```
1.       $T \leftarrow \{\}$ ;  $Y \leftarrow V - \{1\}$ 
2.      for  $y \leftarrow 2$  to  $n$ 
3.  if  $y$  邻接于1 then
4.           $N[y] \leftarrow 1$ 
5.           $key(y) \leftarrow c[1,y]$ 
6.          INSERT( $H,y$ )
7.  else  $key(y) \leftarrow \infty$ 
8.      end if
9. end for
```

NEXT PAGE

华中农业大学信息学院



算法 8.5 MST

```
10. for  $j \leftarrow 1$  to  $n-1$            {查找 $n-1$  条边}
11.    $y \leftarrow \text{DELETMIN}(H)$ 
12.    $T \leftarrow T \cup \{(y, N[y])\}$    {添加边 $(y, N[y])$ 到 $T$ }
13.    $Y \leftarrow Y - \{y\}$            {从 $Y$ 删除顶点 $y$ }
14.   for 每个邻接于 $y$ 的顶点 $w \in Y$ 
15.     if  $c[y, w] < \text{key}(w)$  then
16.        $N[w] \leftarrow y$ 
17.        $\text{key}(w) \leftarrow c[y, w]$ 
18.     end if
19.     if  $w \notin H$  then INSERT( $H, w$ )
20.     else SIFTUP( $H, H^{-1}(w)$ )
21.   end for
22. end for
```

华中农业大学信息学院



算法分析

- 定理

- 给出一个含权无向图 G ，算法MST在 $O(m \log n)$ 时间内找出最小耗费生成树。如果图是稠密的，即如果对于某个 $\epsilon > 0$, $m \geq n^{1+\epsilon}$ ，那么它可以被进一步改进为在 $O(m/\epsilon)$ 时间内运行。

华中农业大学信息学院



8.5 文件压缩

? 问题描述:

- 假设有一个字符串文件, 我们希望用这样一种方法尽可能多地压缩文件, 但源文件能够很容易地被重建。
- 设文件中的字符集是 $C = \{c_1, c_2, \dots, c_n\}$, 又设 $f(c_i), 1 \leq i \leq n$, 是文件中字符 c_i 的频度, 即文件中 c_i 出现的次数。用定长比特数表示每个字符, 称为字符的编码, 文件的大小取决于文件中的字符数。

华中农业大学信息学院



变长编码

- 然而由于有些字符的频度可能远大于另外一些字符的频度，所以用变长的编码是有道理的。从直观上说，那些频度大的字符将被赋予短的编码，而长的编码可以赋给那些频度小的字符。当编码在长度上变化时，我们规定一个字符的编码必须不能是另一个字符编码的前缀(即词头)，这种码称为前缀码。
- 例如，如果我们把编码10和101赋予字符“a”和“b”就会存在二义性，不清楚10究竟是“a”的编码还是字符“b”编码的前缀。

华中农业大学信息学院



Huffman 编码

- 一旦满足前缀约束，编码就变得没有二义性了，就可以扫描比特序列直到找到某个字符的编码。
- 对给定比特序列进行分析的一种方法是用一棵满二叉树，它的每一个内部节点都恰好有两个分支，标记为0和1，这棵树中的叶子对应于字符，从根到叶的每一条路径上的0和1的序列对应于一个字符的编码。
- Huffman 编码：
 - 这个算法由Huffman提出。由算法构造的编码满足前缀约束，并且最小化压缩文件的大小。

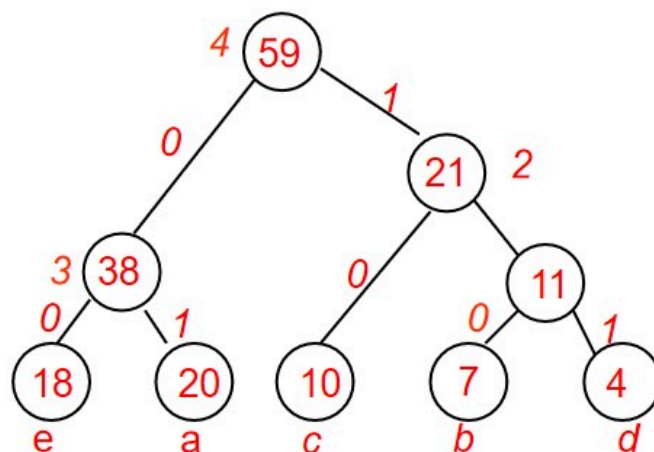
华中农业大学信息学院



例子

- 文件由字母a, b, c, d和e组成, 频度为:

$$f(a)=20, f(b)=7, f(c)=10, f(d)=4 \text{ and } f(e)=18$$



华中农业大学信息学院



算法8.6 HUFFMAN

输入: n 个字符的集合 $C=\{c_1, c_2, \dots, c_n\}$ 和它们的频度 $\{f(c_1), f(c_2), \dots, f(c_n)\}$.

输出: C 的 Huffman 树 (V, T)

1. 根据频度将所有字符插入最小堆 H
2. $V \leftarrow C; T = \{\}$
3. **for** $j \leftarrow 1$ **to** $n-1$
4. $c \leftarrow \text{DELETEMIN}(H)$
5. $c' \leftarrow \text{DELETEMIN}(H)$
6. $f(v) \leftarrow f(c) + f(c')$ $\{v \text{ 是一个新节点}\}$
7. $\text{INSERT}(H, v)$
8. $V = V \cup \{v\}$ $\{\text{添加 } v \text{ 到 } V\}$
9. $T = T \cup \{(v, c), (v, c')\}$ $\{\text{使 } c \text{ 和 } c' \text{ 成为 } T \text{ 中 } v \text{ 的孩子}\}$
10. **end for**

华中农业大学信息学院



算法分析

- 把所有字符插入堆中需要时间 $\Theta(n)$ 见定理4.1)。
- 从堆中删除两个元素和加一个新元素需要的时间是 $O(\log n)$ 。
- 因为这被重复 $n-1$ 次，for循环需要的所有时间是 $O(n \log n)$ ，
- 就得到算法的时间复杂度是 $O(n \log n)$

华中农业大学信息学院